Volume 03 | Runtime Forecasting, Features, and Model Training

Connect the saved estimator to generated labels and later-date evaluation.

1. Two separate computations

Offline training loads millions of generated rows, builds features, splits by date, trains baselines and estimator, scores holdout and saves model/metadata. Runtime loads those artifacts once and creates one feature row per requested hour. It does not train on an API request and does not need to load the giant training CSV during ordinary prediction.

2. Artifact contracts

`app/forecasting.py` eagerly calls `joblib.load` for `footfall_forecaster.joblib` and `forecaster_metadata.joblib`, reads cleaned stations into an indexed DataFrame, and optionally reads metrics JSON. Metadata contains `feature_cols`, station/line IDs, typology IDs and history aggregates. If model and metadata disagree, runtime feature selection or predict can fail; artifact versioning and schema checks are appropriate safeguards. Only trusted joblib files should be loaded.

3. Serving feature construction

`get_station_info` reads primary line, all lines, interchange flag, coordinates, or returns unknown defaults. `predict_hourly_footfall` gates hours outside 5-23 to zero; computes weekday/weekend, cyclic hour/day/month features, holiday/workday context, station/line/typology indices, line count/interchange flag and training-derived historical means. It constructs a DataFrame in metadata's exact feature column order, calls model predict, clips below zero and rounds.

The 23 features are Hour, DayOfWeek, Month, DayOfMonth, DayOfYear, IsWeekend, IsHoliday, IsWorkday, HolidayType, Hour_Sin, Hour_Cos, Day_Sin, Day_Cos, Month_Sin, Month_Cos, Station_Idx, Line_Idx, Typology_Idx, Is_Interchange, Num_Lines, Hist_Station_Mean, Hist_Hour_Day_Mean, Hist_Station_Month_Mean.

4. Interpolation and daily curve

Minute interpolation linearly blends forecast for hour h and h+1 based on minute/60. It is a display estimate, not an estimator trained at minute resolution. Full-day profile loops hours 5..23 inclusive, so 19 points. It identifies maximum point and sums hourly estimates; summing hourly slot values represents a daily total proxy under generated scenario semantics, not unique riders.

5. Training data and features

Pipeline step 03 reads synthetic data and station metadata, merges interchange attributes, applies typology encodings, derives calendar and cyclic features, assigns station/line indices, and splits before 2026-10-01. It computes training historical means and prior-only rows. Holdout features use aggregates from training period only. The target called Footfall is synthetic scenario output.

6. Models and scores

Baselines: station overall mean (MAE 298.74), station/day/hour historical mean (MAE 60.25), Ridge (MAE 53.67). HistGradientBoostingRegressor MAE 26.58, RMSE 51.10, R2 0.9946, MAPE 5.18%. Holdout: 438,615 synthetic rows; total 3,361,176, train 2,922,561. Chronological split is more relevant to future dates than random split, but synthetic generator may be very learnable and does not establish real-world accuracy.

7. Leakage and validity audit

Training rows use prior-date aggregates; test rows use only training history. This is a useful leakage defense. Still ask whether the feature would exist at prediction time, whether occasion labels are known ahead, whether generator rules leak through labels, and whether holdout covers realistic shift. Validate on observed data before making operational claims. Report errors by station, hour, weekend/holiday, and demand level, and compare against simple baselines.

Exercises

Explain why R2 is not accuracy; work a tiny MAE/RMSE example; show hour cyclical encoding; compare minute interpolation with 19 model calls; locate split and history features; identify one source of synthetic-to-real domain gap.